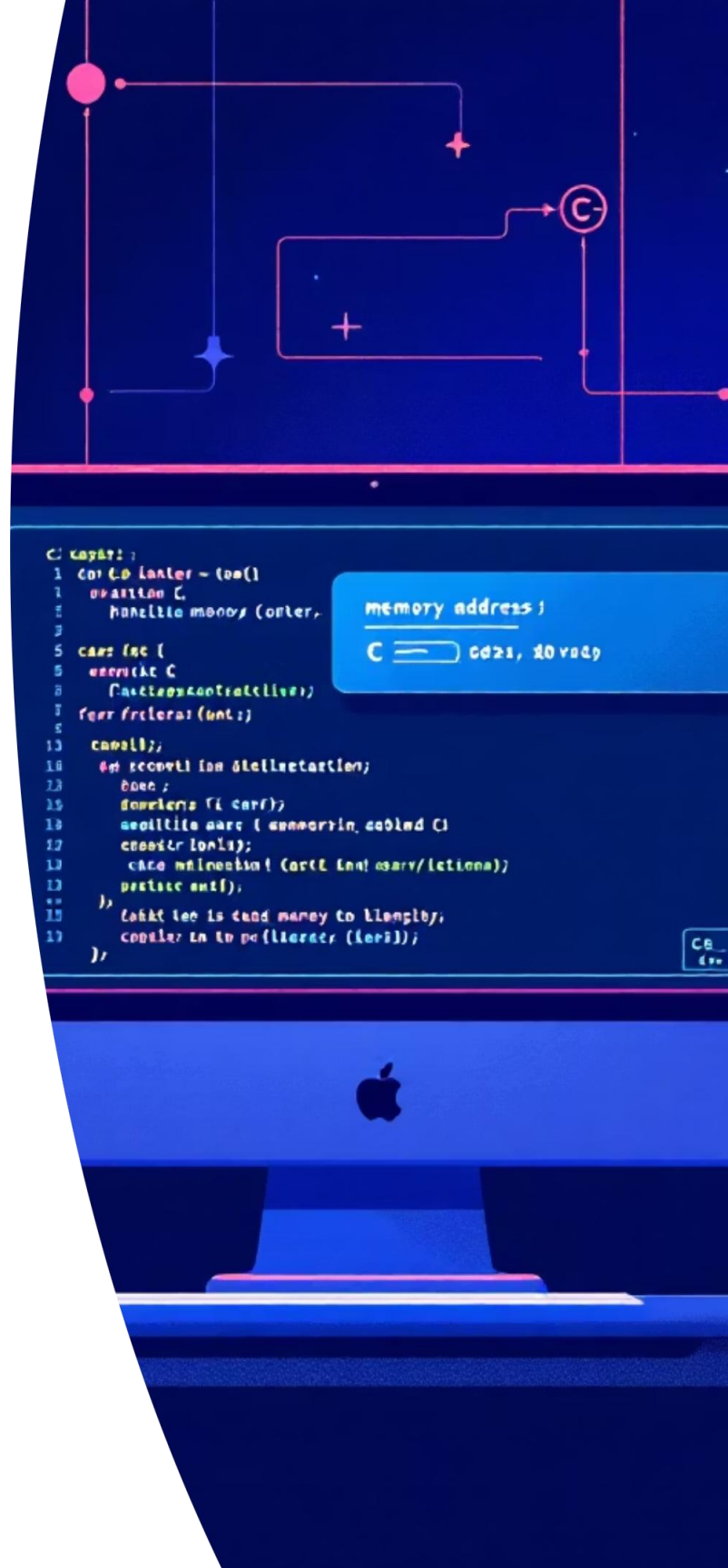
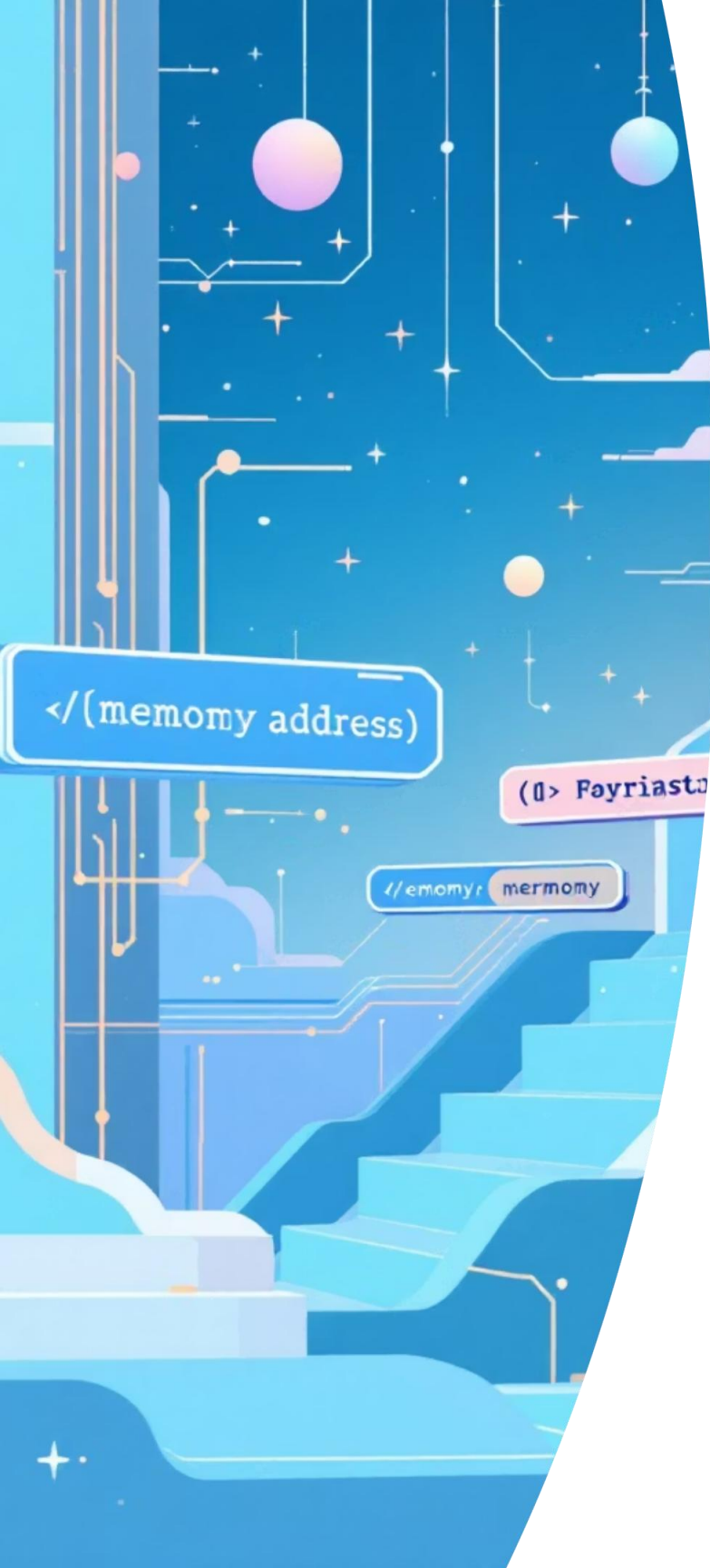


Function & Structure Pointers in C

A beginner-friendly guide to mastering pointers, function pointers, and structure pointers in C programming.





Introduction to Pointers

What's a Pointer?

A pointer is a variable that stores the **memory address** of another variable. Think of it as a signpost pointing to where data lives in memory.

Key Operators

- **& (ampersand)** – gets the address of a variable
- *** (asterisk)** – accesses the value at that address

Quick Example

```
int age = 18;  
int *ptr = &age;  
printf("%d", *ptr);  
// Output: 18
```

Here, **ptr** points to **age**, and ***ptr** retrieves the value 18.

Function Pointers – Core Concept



What Are They?

Function pointers store the address of a function, allowing you to call functions dynamically at runtime.



Declaration Syntax

Declare with return type, pointer name in parentheses, and parameter types.

Simple Example

```
int add(int a, int b) {  
    return a + b;  
}  
  
int (*fp)(int, int) = add;  
int result = fp(12, 3);  
// result is 15
```

Breaking it down:

1. `fp` is a pointer to a function
2. We assign it the address of `add`
3. Call it just like a regular function!

Function Pointers: Dynamic Operation Selection

Function pointers allow for highly flexible code by letting you choose which function to execute at runtime. Observe how a single pointer can be reassigned to different arithmetic operations.

```
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }

int main() {
    int (*op_ptr)(int, int); // Declare function pointer

    op_ptr = add; // Point to add()
    printf("Add: %d\n", op_ptr(10, 5)); // Calls add(10, 5) -> 15

    op_ptr = subtract; // Point to subtract()
    printf("Subtract: %d\n", op_ptr(10, 5)); // Calls subtract(10, 5) -> 5

    op_ptr = multiply; // Point to multiply()
    printf("Multiply: %d\n", op_ptr(10, 5)); // Calls multiply(10, 5) -> 50

    return 0;
}
```

The same function pointer, `op_ptr`, is used to perform different calculations by simply reassigning it to the desired function.



Function Definitions

Standard functions `add`, `subtract`, and `multiply` are defined, each with the same signature.



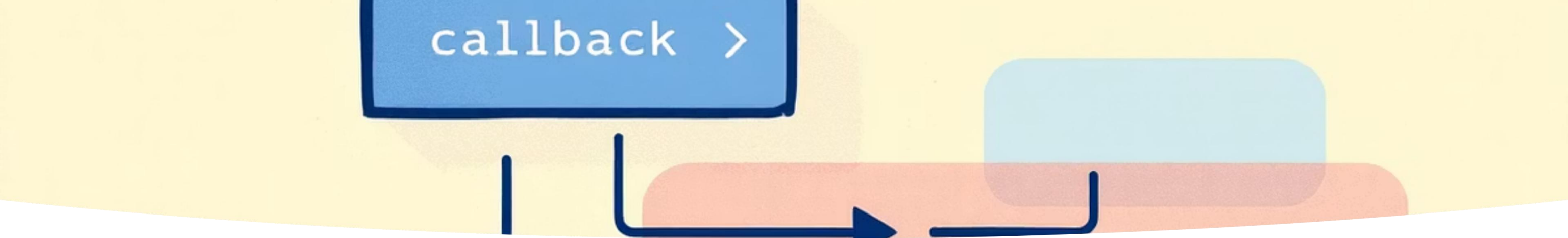
Pointer Declaration

A function pointer `op_ptr` is declared, capable of pointing to any function that takes two integers and returns an integer.



Dynamic Reassignment

`op_ptr` is sequentially assigned to `add`, `subtract`, and `multiply`, demonstrating dynamic function calling.



callback >

Function Pointer Use Case: Callbacks

Function pointers shine when implementing callbacks – functions passed as arguments to other functions. This enables flexible, reusable code patterns.

Callback Demo

```
void execute(int (*operation)(int, int), int x, int y) {  
    printf("Result: %d\n", operation(x, y));  
}
```

```
execute(add, 5, 3); // Passes 'add' function  
// Output: Result: 8
```

📌 **Real-world use:** Callbacks power event handlers, sorting algorithms (like `qsort`), and custom behaviour injection in libraries.

Structure Pointers – Core Concept

01

The -> Operator

Access struct members through a pointer using the arrow operator `->` instead of dot notation.

Structure Pointer Example

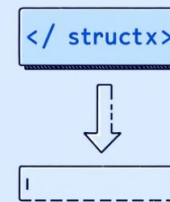
```
struct Person {  
    int age;  
};  
  
struct Person p = {18};  
struct Person *ptr = &p;  
ptr->age = 19;
```

02

Syntax Sugar

`ptr->member` is shorthand for `(*ptr).member` – much cleaner and easier to read!

After this code runs, the person's age is updated to **19** through the pointer.



Practical Structure Pointers: Student Data

Let's apply structure pointers to a common scenario: managing student records. This example demonstrates creating a `Student` structure and manipulating its data using a pointer.

```
#include <stdio.h>
#include <string.h> // For strcpy

struct Student {
    char name[50];
    char college[50];
    int roll_no;
};

int main() {
    // Declare a Student variable
    struct Student s1;

    // Initialize s1
    strcpy(s1.name, "Alice Smith");
    strcpy(s1.college, "DDU"); // College is always DDU
    s1.roll_no = 101;

    // Declare a pointer to Student and point it to s1
    struct Student *s_ptr = &s1;

    // Access and print members using the pointer
    printf("Name: %s\n", s_ptr->name);
    printf("College: %s\n", s_ptr->college);
    printf("Roll No: %d\n", s_ptr->roll_no);

    // Modify a member using the pointer
    s_ptr->roll_no = 102;
    printf("New Roll No: %d\n", s1.roll_no); // Verifying change via original struct
    return 0;
}
```

Dissecting the Example:

- **Structure Definition:** We define `struct Student` containing a name, a fixed college name, and a roll number.
- **Instance Creation:** A variable `s1` of type `struct Student` is declared and initialised with basic data.
- **Pointer Declaration:** A pointer `s_ptr` is created to store the memory address of `s1`.
- **Access & Modification:** We use the `->` operator to access and print the student's details, and then to update the roll number directly through the pointer.

📌 Using structure pointers is especially powerful when dealing with arrays of structures or dynamically allocated memory, simplifying data manipulation.

Mini Quiz & Solutions

1

Question 1: Function Pointer Call

```
int sum(int a, int b) { return
a + b; }
int (*op_func)(int, int) =
sum;
int result = op_func(7, 3);
printf("%d", result);
```

Solution: Output is **10**. The function pointer `op_func` calls `sum(7, 3)`.

2

Question 2: Function Pointer Reassignment

```
int square(int n) { return
n*n; }
int (*fp)(int) = square;
printf("%d", fp(5));
```

Solution: Output is **25**. The function pointer calls `square(5)`.

3

Question 3: Struct Pointer

```
struct Data { int val; };
struct Data d = {7};
struct Data *ptr = &d;
ptr->val += 3;
printf("%d", d.val);
```

Solution: Output is **10**. The arrow operator modifies the struct's member (`7 + 3`).

Congratulations! You've mastered the fundamentals of pointers, function pointers, and structure pointers in C. Keep practising with real code to solidify these concepts!